

A RSA

I find that I don't understand things unless I try to program them.

— Donald E. Knuth, The Art of Computer Programming, Volume 4

Python kann beliebig große (Prim)Zahlen – wir nutzen das zum Verschlüsseln. Dazu brauchen wir zuerst einige Hilfsfunktionen.

Wichtig:

- diese Funktionen kommen in eine oder mehrere Dateien
- mittels `import` kann man in anderen Dateien darauf zugreifen

A.1 Primzahlen Test: Rabin-Miller

Ein vollständiger Test `is_prime()` ist für sehr große Zahlen nicht durchführbar. Es gibt Näherungslösungen *ist ziemlich wahrscheinlich eine Primzahl*.

A.1.1 Vorbereitung

Datei: `fermat.py`

Primzahlen

Berechne (in Python) für jede der Primzahlen $p = 2$ bis $p = 11$ und $p = 997$:

- für jeweils jedes Element a (ohne 0) von Z_p (d.h. Restklasse p bzw. $\dots \bmod p$, $a = 1 \dots p - 1$)
 – den Wert a^{p-1} in Z_p .

Erkenntnis?¹

Was passiert bei *Nicht-Primzahlen*?

Rechnung wie oben: Teste (als p) die Zahlen 9, 15, 21, 551 ... 570, 6601, 8911². Wie oft kommt 1 als Ergebnis vor (absolut und in %), wie viele verschiedene Ergebnisse gibt es³?

Ergebnisse

```

2 -> 100 % -> res[1]=1, len(res)=1 - [(1, 1)]
3 -> 100 % -> res[1]=2, len(res)=1 - [(1, 2)]
5 -> 100 % -> res[1]=4, len(res)=1 - [(1, 4)]
7 -> 100 % -> res[1]=6, len(res)=1 - [(1, 6)]
11 -> 100 % -> res[1]=10, len(res)=1 - [(1, 10)]
997 -> 100 % -> res[1]=996, len(res)=1 - [(1, 996)]
9 -> 25 % -> res[1]=2, len(res)=4 - [(1, 2), (4, 2), (0, 2), (7, 2)]
15 -> 28 % -> res[1]=4, len(res)=5 - [(1, 4), (4, 4), (9, 2), (10, 2), (6, 2)]
21 -> 20 % -> res[1]=4, len(res)=7 - [(1, 4), (4, 4), (16, 4), (9, 2), (15, 2), (7, 2), (18, 2)]
551 -> 0 % -> res[1]=4, len(res)=149 - [(1, 4), (245, 4), (35, 4), (517, 4), (480, 4), (310, 4), (8, 4), (424, 4), (288, 4), (141, 36), (204, 36), (176, 36), (225, 36), (64, 36), (1, 36), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1), (289, 8), (271, 8), (379, 8), (64, 8), (121, 8), (8, 6), (64, 6), (216, 6), (512, 6), (444, 6), (60, 6), (556, 6)]
552 -> 0 % -> res[1]=1, len(res)=345 - [(416, 4), (280, 4), (144, 4), (8, 4), (424, 4), (288, 4), (141, 36), (204, 36), (176, 36), (225, 36), (64, 36), (1, 36), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1), (289, 8), (271, 8), (379, 8), (64, 8), (121, 8), (8, 6), (64, 6), (216, 6), (512, 6), (444, 6), (60, 6), (556, 6)]
553 -> 6 % -> res[1]=36, len(res)=27 - [(1, 36), (64, 36), (176, 36), (225, 36), (141, 36), (204, 36), (176, 36), (225, 36), (141, 36), (204, 36), (1, 36), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1), (289, 8), (271, 8), (379, 8), (64, 8), (121, 8), (8, 6), (64, 6), (216, 6), (512, 6), (444, 6), (60, 6), (556, 6)]
554 -> 0 % -> res[1]=1, len(res)=553 - [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1), (289, 8), (271, 8), (379, 8), (64, 8), (121, 8), (8, 6), (64, 6), (216, 6), (512, 6), (444, 6), (60, 6), (556, 6)]
555 -> 1 % -> res[1]=8, len(res)=113 - [(1, 8), (289, 8), (271, 8), (379, 8), (64, 8), (121, 8), (8, 6), (64, 6), (216, 6), (512, 6), (444, 6), (60, 6), (556, 6)]
556 -> 0 % -> res[1]=3, len(res)=141 - [(8, 6), (64, 6), (216, 6), (512, 6), (444, 6), (60, 6), (556, 6)]
557 -> 100 % -> res[1]=556, len(res)=1 - [(1, 556)]

```

¹Kleiner Fermatscher Satz

²siehe <https://de.wikipedia.org/wiki/Carmichael-Zahl>

³Tip: `from collections import Counter`

```

558 -> 0 % -> res[1]=1, len(res)=434 - [(549, 3), (522, 3), (81, 3), (414, 3), (333, 3), (324, 3),
559 -> 6 % -> res[1]=36, len(res)=23 - [(1, 36), (441, 36), (391, 36), (508, 36), (428, 36), (259,
560 -> 0 % -> res[1]=1, len(res)=315 - [(128, 8), (144, 8), (496, 8), (512, 8), (80, 8), (208, 8),
561 -> 57 % -> res[1]=320, len(res)=7 - [(1, 320), (375, 160), (154, 32), (34, 20), (528, 16), (408
562 -> 0 % -> res[1]=1, len(res)=561 - [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8
563 -> 100 % -> res[1]=562, len(res)=1 - [(1, 562)]
564 -> 0 % -> res[1]=1, len(res)=423 - [(356, 2), (400, 2), (108, 2), (272, 2), (304, 2), (96, 2),
565 -> 2 % -> res[1]=16, len(res)=57 - [(1, 16), (16, 16), (81, 16), (256, 16), (166, 16), (141, 1
566 -> 0 % -> res[1]=1, len(res)=565 - [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8
567 -> 0 % -> res[1]=4, len(res)=112 - [(324, 54), (162, 54), (81, 54), (0, 26), (1, 4), (445, 4),
568 -> 1 % -> res[1]=7, len(res)=55 - [(128, 28), (480, 28), (96, 28), (360, 28), (88, 28), (72, 2
569 -> 100 % -> res[1]=568, len(res)=1 - [(1, 568)]
6601 -> 80 % -> res[1]=5280, len(res)=7 - [(1, 5280), (3773, 880), (575, 240), (2255, 132), (4347, 4
8911 -> 80 % -> res[1]=7128, len(res)=7 - [(1, 7128), (1274, 1188), (7505, 396), (134, 108), (8778,

```

A.1.2 Miller Rabin

def is_prim(n)

was nicht geht:

- Schleife: bis sqrt(n)
 - $n \sim 1000$ Bits = ca. 300 Stellen dez., $\sqrt{n} \sim 150$ Stellen dez.

Aber:

- n ist ja ungerade⁴ Daher ist $(n-1)$ gerade, also gilt $(n-1) = 2^s * d \rightarrow$ Unterprogramm

Nach dem kleinen Satz von Fermat gilt:

- Entweder
 - $a^d \equiv 1 \pmod{n} \rightarrow$ Primzahl (wahrscheinlich) $\rightarrow a^{d \cdot 2^r} \equiv 1$ für alle $0 \leq r < s$
- Oder es gilt für a (wobei $a < n$), für alle r (wobei $0 \leq r < s$):
 - $a^{d \cdot 2^r} \equiv -1 \pmod{n}$ für ein $r \rightarrow$ Primzahl (wahrscheinlich)
 - $a^{d \cdot 2^r} \not\equiv -1 \pmod{n} = n-1 \pmod{n}$ für alle $r \rightarrow$ keine Primzahl

Bsp: $n=221$ is prime?

We write $n-1 = 220$ as $2^2 * 55$, so that we have $s = 2$ and $d = 55$

Random $a = 174$. We proceed to compute:

$$a^{d \cdot 2^0} \pmod{n} = 174^{55} = 1.699 * 10^{123} \pmod{221} \equiv 47 \neq \{1, n-1\}$$

$$a^{d \cdot 2^1} \pmod{n} = 174^{55 \cdot 2} = 2.88 * 10^{246} \pmod{221} \equiv 220 = n-1 \rightarrow \text{maybe prime}$$

by the way: $174^{55 \cdot 2} = (174^{55})^2$ and $47^2 \pmod{221} \equiv 220$

Since $220 \equiv -1 \pmod{n}$, either 221 is prime, or 174 is a strong liar for 221. ⁵

We try another random a , this time choosing $a = 137$:

$$a^{d \cdot 2^0} \pmod{n} = \dots \pmod{221} \equiv 188 \neq 1, n-1$$

$$a^{d \cdot 2^1} \pmod{n} = \dots \pmod{221} \equiv 205 \neq n-1 \rightarrow \text{not prime}$$

Maybe bedeutet Wahrscheinlichkeit beträgt höchstens $\frac{1}{4} \rightarrow$ mit vielen Zahlen testen

Idee: Test in 2 Phasen:

⁴die einzige gerade Primzahl ist ja 2

⁵all liars: 21 47 174 200

- 1) eine (fixe) Liste der ersten z.B. 100 Primzahlen ist hilfreich
 - damit kann man die Teilbarkeit leicht testen
- 2) alle Zahlen die nicht teilbar sind werden zusätzlich mit `is_prim_millerrabin()` getestet, 20x dann ist die Fehlerwahrscheinlichkeit sehr gering.

A.1.3 Implementierung

Datei: `millerrabin.py`

siehe Pseudocode unter http://en.wikipedia.org/wiki/Miller-Rabin_primality_test

Input #1: `n > 3`, an odd integer `to` be tested `for` primality

Input #2: `k`, the number of rounds of testing `to` perform

Output: "composite" `if` `n` is found `to` be composite, "probably prime" otherwise

write `n` as `d·2r + 1` with `d` odd (by factoring out powers of 2 from `n - 1`)

WitnessLoop: `repeat` `k` times:

pick a `random` integer `a` in the range `[2, n - 2]`

`x ← ad mod n`

`if` `x = 1` or `x = n - 1` then

`continue` WitnessLoop

`repeat` `r - 1` times:

`x ← x2 mod n`

`if` `x = n - 1` then

`continue` WitnessLoop

`return` "composite"

`return` "probably prime"

Tipps für eine Funktion `is_prim_millerrabin(number, anzahl=20)`

- Führe den Algorithmus mindestens `anzahl`-mal durch
 - der Test findet Primzahlen nur mit einer gewissen Wahrscheinlichkeit, nach `anzahl` Tests ist die Fehlerwahrscheinlichkeit $\frac{1}{4^{\text{anzahl}}}$ und daher nahezu null.
- `pow(x,y,z)` rechnet mit Restklasse `z`: *zufälligerweise* gibt es das schon in Python (weiter unten implementieren wir das selber)!
- die *innere* Schleife(`repeat`)
 - in ein eigenes Unterprogramm
 - oder *tricksen* mit `else` bei der Schleife

Tipps für eine Funktion `is_prime(number)`:

- eine (fixe) Liste der ersten z.B. 100 Primzahlen ist hilfreich
 - damit kann man sehr schnell die Teilbarkeit testen
 - alle Zahlen die nicht durch diese 100 Primzahlen teilbar sind, werden anschließend mit `is_prim_millerrabin()` getestet. Mit `anzahl=20` handelt es bei `True` ziemlich sicher um eine Primzahl.

Eine Funktion `generate_prime(bits)`:

- Schleife
 - Erzeuge eine zufällige Zahl mit genügend Bits⁶
 - * sinnvoll: im Intervall `max/2 ... max` – sonst hat man führende 0-er und damit effektiv weniger Bits
 - * Trick: die Zahl ist ungerade und mindestens `max/2` groß – Binärdarstellung: welche Bits müssen gesetzt sein? Das kann man erzwingen.
 - bis es (wahrscheinlich) eine Primzahl ist dh. `is_prime()` liefert `true`.

⁶Alternativ: weiterzählen, aber es könnte große Lücken zwischen den Primzahlen geben...

Achtung:

- die Verschlüsselung ist nur so gut wie die verwendeten Zufallszahlen. *Pseudo*zufallszahlen sind auch nur *pseudosicher* – nur weil man selbst die Verschlüsselung nicht knacken kann bedeutet das nicht **niemand** kann es knacken. Python bietet mit `random.SystemRandom` bzw. `os.urandom()` auch *bessere* Zufallszahlen an.
- Lesetipp: <https://eli.thegreenplace.net/2019/rsa-theory-and-implementation/>
- Wir implementieren *Textbook-RSA* – da gibt es einige Probleme.
 - <https://crypto.stackexchange.com/questions/20085/which-attacks-are-possible-against-raw-textbook-rsa>
 - <http://www.ams.org/notices/199902/boneh.pdf>

Aufgabe/Test: bestimme die erste Primzahl $> 2^{512}$ (Lösung: 134...171)

Magic: Ist 24566544301293569 prim? Enthält diese Zahl eine Botschaft ? Stelle die Zahl binär mit 12 Zeichen/Zeile dar...

Bestimme die nächsthöhere Primzahl! Enthält diese noch immer diese Botschaft?

A.2 Schnelle Exponentiation: `pow()`

Datei: `pow.py`

Die Python-Funktion `pow(a,b,n)` schafft es, Potenzen großer Zahlen zu berechnen und ggf. sogar das Ergebnis modulo `n` zu bilden. Jetzt sind wir am Zug.

Idee:

- Rekursiv⁷: x^b
 - ungerades `b`:

$$x^b = x^{(b-1)/2} \cdot x^{(b-1)/2} \cdot x$$

- gerades `b`:

$$x^b = x^{b/2} \cdot x^{b/2}$$

- Iterativ:
 - bestimme die *Binärdarstellung* von `b`. Beispiel:

$$x^5 = x^{101b} = x^4 \cdot x^1$$

- rechne beginnend mit dem letzten Bit von `b`.
 - * die entsprechende Potenz von `x` durch quadrieren berechnen
 - * für jedes Bit 1: die entsprechende Potenz von `x` kommt zum Ergebnis dazu
- Trick: alle Zwischenergebnisse bereits `mod n` berechnen, dann bleiben die Zahlen klein

Aufgabe:

- Implementiere mindestens eine eigene Version dieser Funktion (iterativ oder rekursiv)
- Teste deine Funktion(en) gegen das mitgelieferte `pow()`. Vergleiche dazu die Ergebnisse der verschiedenen Funktionen
 - richtige Ergebnisse
 - Rechenzeit bei großen Werten (besonders für die Hochzahl).

A.3 Keys erzeugen `generate_keys(number_of_bits)`

Datei: `rsa.py`

Parameter: gewünschte Länge des Keys in bits bzw. welche Blockgröße soll beim Verschlüsseln verwendet werden⁸.

⁷Zwischenergebnis speichern! nicht zwei rekursive Aufrufe mit dem selben Wert

⁸da ist ein kleiner Unterschied, wir nehmen die zweite Definition

Vorgehen:

- zwei Primzahlen p , q erzeugen, etwa mit der *halben* Anzahl an Bits⁹
 - die *halbe* Anzahl ist natürlich eine Einschränkung, andererseits gibt es¹⁰ ca. $2 \cdot 10^{151}$ Primzahlen mit einer Länge von 512 Bit.
- bestimme $n = p \cdot q$
 - n muss mehr als die erforderliche Anzahl an Bits haben!¹¹
 - Kontrolle: $n.\text{bit_length}() > \text{Blockgröße}$
- bestimme e : *große* Zufallszahl. Diese muss teilerfremd zu $\varphi(n) = (p-1) \cdot (q-1)$ sein!¹²
- bestimme $d = \text{ModInverse}(e, (p-1) \cdot (q-1))$ = die *multiplikativ Inverse* zu $e \bmod \varphi(n)$.
 - Es muss daher folgende Gleichung gelten: $e \cdot d = 1 \bmod \varphi(n)$.
 - Tipp: Dokumentation zu `pow()`¹³ oder Extended Euclidean algorithm bzw. fast mod inverse
- Ergebnis ist ein Tupel: (private key, public key)
 - jeder Key besteht aus (e bzw. d , n , keylen)¹⁴

Ultimativer Test – als doctest:

```
for x in [ ...a lot of numbers, small and large...]:
    c = pow(x, e, n)
    y = pow(c, d, n)
    assert x == y
```

A.4 Nachricht in Integer-zahlen file2ints()

- Python3 kennt *binäre Dateien*
 - einfach ein “b” beim Öffnen angeben
 - bei `file.read()` kann man die Anzahl der Bytes angeben
 - Achtung: Wie erkennt man das Dateiende?
- es gibt `int.from_bytes()` und `int.to_bytes()`
 - Wozu braucht man den Parameter `byteorder`?

Lese die Datei und liefere die Ergebnisse (*Byteblöcke* als *int*) idealerweise als Generator.

Jetzt braucht man *nur* noch die vielen Zahlen mittels RSA verschlüsseln – aber das ist ein anderes Unterprogramm. Und das Schreiben der Zahlen in eine Datei ist auch ein Kandidat für ein Unterprogramm.

⁹ p und q müssen ungleich sein – Achtung daher bei kleiner Blockgröße! Außerdem sollten sie nicht zu nahe beieinander liegen – siehe Abschnitt **Hackerangriff** weiter unten.

¹⁰<https://www.heise.de/select/ix/2023/2/2233514273520115348>

¹¹es kann auch p kleiner/größer sein, dann muss man q dazu passend erzeugen

¹²sonst existiert keine multiplikativ Inverse zu e

¹³ab Python 3.8: <https://docs.python.org/3/library/functions.html#pow>

¹⁴man sollte sich die Blockgröße / Länge des Keys merken, damit ist später das Entschlüsseln einfacher

A.5 Endprodukt

Ein Programm – aufgerufen mit verschiedenen Optionen (Tipp: `import argparse`, siehe unten), Minimalversion
`rsa.py [-h] [-v] (-k KEYGEN | -e ENCRYPT | -d DECRYPT)`

optional arguments:

```
-h, --help            show this help message and exit
-v, --verbosity        increase output verbosity
-k KEYGEN, --keygen KEYGEN
                        generate new keys with the given length
-e ENCRYPT, --encrypt ENCRYPT
                        encrypt file
-d DECRYPT, --decrypt DECRYPT
                        decrypt file
```

Je eine Option

- Erzeugen des Schlüsselpaares
 - die Schlüssel kommen in Dateien mit fixen Dateinamen
 - speichere auch die Länge des Schlüssels (Blockgröße)
 - openssl speichert es so:

```
openssl genrsa -out private_key.pem 1024
openssl rsa -text -in private_key.pem
```
- Verschlüsseln einer Datei
 - Lese Schlüssel (Welchen?)
 - Zerlege die Eingabedatei in Blöcke und verschlüssele jeden Block
 - * Achtung: Für das RSA-Verfahren muss die Länge der Blöcke kleiner als die Länge des Schlüssels sein.
 - Speichere die verschlüsselten Daten in eine neue Datei
 - * Vereinfachung: Speichere die Zahlen als Text (mit einem Zeilenwechsel oder einem anderen sinnvollen Trennzeichen).
 - * Besser: speichere die Zahlen als `bytes`
 - * Alternative: Speichere die Daten in einer Liste und verwende `pickle`.

Tipp: wenn `encrypt/decrypt` als Parameter “etwas iterierbares” haben kann man auch mit Listen testen!
- Entschlüsseln einer Datei
 - Lese Schlüssel (Welchen?)
 - Lese die Zahlen und *entschlüsse* den Block
 - Schreibe die Bytes wieder in eine neue Datei

Weiters (optional, Bonus):

- Unterschreiben einer Datei
- Kontrolle der Unterschrift

A.5.1 Muster

```
parser = argparse.ArgumentParser()
parser.add_argument("-v", "--verbosity", help="increase output verbosity", action="store_true")
group = parser.add_mutually_exclusive_group(required=True)
group.add_argument("-k", "--keygen", help="generate new keys with the given length", type=int)
...

args = parser.parse_args()
```

```
if args.verbosity: # überall wo man eine bedingte Ausgabe braucht - besser: loglevel setzen
    print("verbosity turned on")
    # logging.basicConfig(level=...)

if args.keygen:
    print("keygen: ", args.keygen)

oder

    parser.add_argument("-l", "--loglevel", default="WARNING",
                        choices=['DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL'],
                        help="Set the logging level")

    ...
    # logging.basicConfig(level=getattr(logging, args.logLevel))
```

Alternativen (zusätzlich implementieren für Experten) – Liste ist unvollständig:

- <http://docopt.org/>
- <http://click.pocoo.org/5/>

Achtung: `argparse` kann endlos viel, bitte Hilfe unbedingt lesen!

Tipps zum Testen:

- mit kleiner `keysize` und kleinen Dateien testen.
- `sys.args` kann man
 - in PyCharm angeben (*Parameter*)
 - zu Testzwecken auch bei `parser.parse_args()` mitgeben

Logging:

- entweder wie oben: `if` und `print`
- oder gleich *echtes* logging – kommt nächstes Jahr auf jeden Fall: <https://docs.python.org/3/howto/logging-cookbook.html>

A.5.2 Anmerkung – Wieviele Bytes werden als Block gelesen bzw. geschrieben?

Verschlüsseln:

- 1) Klartext aus Datei lesen
- 2) verschlüsselte Nachricht schreiben

Entschlüsseln:

- 3) verschlüsselte Nachricht lesen
- 4) Klartext schreiben

bei 1) und 4):

- die angegebene (gewünschte) Bitlänge – Blockgröße
- Wert ist kleiner/kürzer/*weniger Bits* als die Länge von `N (n.bit_length())`
 - bei 4) wird ... % `N` geschrieben, das muss alle möglichen Werte umfassen. maximal ist das binär 11...111
- Achtung: Bits vs. Bytes

bei 2) und 3):

- einfach: als Text mit Trennzeichen (Leerzeichen oder Zeilenwechsel)
- oder binär:
 - das maximale Ergebnis der Verschlüsselung ist `N-1` (wegen ... % `N`)
 - man muss also `N-1` speichern können dh. die Anzahl der Bytes ist groß genug/*mehr* als die Länge von `N-1`

A.5.3 Anmerkung – der letzte Block?

Wir schreiben immer ganze Blöcke d.h. die entschlüsselte Datei hat eine *falsche* Länge und viele 0-Bytes am Ende.

Lösung:

- ignorieren: bei vielen Dateiformaten ist es egal, geht aber nur bei *richtiger* Byteorder
- Länge extra in der verschlüsselten Datei speichern (ganz am Anfang)
- siehe RFC 2313 (mehrfach obsolete, RFC 8017): Encryption-block formatting bzw. <https://eli.thegreenplace.net/2019/rsa-theory-and-implementation/#practical-considerations>

A.6 Fermat-Angriff

Siehe <https://it.slashdot.org/story/22/03/14/228258/researcher-uses-379-year-old-algorithm-to-crack-crypto-keys-found-in-the-wild>

Das können wir auch: Schreibe das Programm `rsa_attack.py`:

```
python rsa_attack.py
usage: rsa_attack.py [-h] [-v] [-m MAX] modul
rsa_attack.py: error: the following arguments are required: modul
```

```
python rsa_attack.py 1000001000000090000037000001961
(1000000000000037, 10000010000000053)
```

```
python rsa_attack.py -v 1000001000000090000037000001961
```

Es braucht 125 Versuche, um die Faktoren von 1000001000000090000037000001961 zu finden:

```
p = 10000000000000037
q = 10000010000000053
```

Funktionsweise: Der RSA-Modul N ist das Produkt der zweier großer Primzahlen p und q . Weil p und q ungeradzahlig sind, existiert eine ganzzahlige Mitte a zwischen p und q . Daher sind p und q gleich weit von der Mitte a entfernt. Diesen Abstand nennen wir b .

$$N = p \times q = (a - b) \times (a + b) = a^2 - b^2 \quad | \quad + b^2 - N$$

$$b^2 = a^2 - N$$

$$b = \sqrt{a^2 - N}$$

Algorithmus:

- Beginne mit $a = \lceil \sqrt{N} \rceil$
- Tipp: `math.isqrt`
- Ist die Quadratwurzel aus b eine ganze Zahl Treffer!
- Sonst: a erhöhen und nochmals probieren.

Warum ist bei RSA p zufällig, q ist die nächste Primzahl nach p eine ganz schlechte Idee?

* Lesetipps:

- * <https://madhavamathcompetition.com/tag/fermats-factorization-method/> bzw.
- * https://en.wikipedia.org/wiki/Fermat%27s_factorization_method